# ExoHabitAI

## Exoplanet Habitability Predictor

Module 5 & 6 — Backend API Integration + Frontend UI Development
March 2025

| Habitability Parameters | Pre-loaded Exoplanets | Habitability Categories | API Endpoints |
|---|---|---|---|
| **6 Key Parameters** | **28 Real Planets** | **3 Categories** | **4 Endpoints** |

Built with Flask + Scikit-learn + HTML / CSS / JavaScript

# Table of Contents

# 1.
## Project Overview

ExoHabitAI is an AI-powered full-stack web application that predicts the habitability potential of exoplanets using planetary and stellar parameters. It combines a trained Machine Learning model with a rule-based scoring system, served through a Flask REST API and presented through a responsive dark-themed web interface.

- Predicts habitability using 6 key exoplanet parameters
- Weighted rule-based scoring with intelligent penalty system
- 28 real exoplanets pre-loaded across 3 habitability categories
- Real-time API status indicator and session prediction history

# 2.
## Tech Stack

| Layer | Technology | Purpose |
|---|---|---|
| Frontend | HTML5, CSS3, JavaScript, Bootstrap 5.3 | UI layout, forms, and interactivity |
| Styling | Google Fonts (Orbitron), Font Awesome | Space-themed typography and icons |
| Backend | Python 3.11, Flask 2.3, Flask-CORS | REST API server and routing |
| ML Model | Scikit-learn — Random Forest + Pipeline | Habitability classification |
| Data | NumPy, Pandas, Joblib | Feature processing and model I/O |
| Version Control | Git, GitHub | Source control and collaboration |

# 3.
## Folder Structure

```
ExoHabitAI/
■■■ backend/ app.py (Flask API routes) utils.py (validation + scoring)
■■■ frontend/ index.html style.css script.js
■■■ models/ exohabit_model.pkl feature_names.json
■■■ notebooks/ (Jupyter training notebooks)
■■■ requirements.txt README.md
```

# 4.
## Module 5 — Backend API (Flask)

### 4.1 Environment Setup

```
python3 -m venv venv && source venv/bin/activate
pip install flask flask-cors joblib numpy pandas scikit-learn
```

```
cd backend && python app.py ➜ Running on http://127.0.0.1:5000
```

### 4.2 API Endpoints

| Endpoint | Method | Description |
|---|---|---|
| GET / | GET | Welcome message and available endpoints |
| GET /health | GET | API health check and model loaded status |
| POST /predict | POST | Predict habitability of a single exoplanet |
| POST /rank | POST | Rank multiple planets by habitability score |

### 4.3 /predict — Request & Response

**Request Body:**

```
{ "planet_name": "Kepler-452b", "planet_radius": 1.6, "orbital_period": 384.8,
"equilibrium_temperature": 265, "semi_major_axis": 1.05,
"stellar_luminosity": 1.2, "stellar_mass": 1.04 }
```

**Success Response:**

```
{ "status": "success", "planet_name": "Kepler-452b", "prediction": 1,
"habitability_status": "Potentially Habitable", "habitability_score": 82.5,
"confidence": "Moderately Habitable", "message": "Analysis complete" }
```

### 4.4 Input Validation

| Parameter | Required | Valid Range |
|---|---|---|
| planet_radius | Yes | 0.1 – 30 Earth radii |
| orbital_period | Yes | 0.1 – 100,000 days |
| equilibrium_temperature | Yes | 50 – 1,000 Kelvin |
| semi_major_axis | Yes | 0.001 – 100 AU |
| stellar_luminosity | Yes | 0.0001 – 1,000 L |
| stellar_mass | Yes | 0.01 – 150 M |

# 5.
# Module 6 — Frontend UI

### 5.1 Page Sections

| Section | Description |
|---|---|
| Navbar | Logo + real-time API Online / Offline status dot |
| Hero | Animated CSS starfield, headline, and stat badges |
| Input Form (Left) | 7 parameter fields with labels, units, and inline validation |
| Results Panel (Right) | 4 states: Placeholder, Loading, Results, Error |
| Load Sample Data | 28 real planets in Habitable / Borderline / Not Habitable groups |

| | |
|---|---|
| History Table | Session predictions with name, score, status badge, and timestamp |
| How It Works | 3-step process explanation cards |

### 5.2 Frontend → Backend Connection

JavaScript Fetch API sends a POST request to the /predict endpoint:

```
const response = await fetch('http://127.0.0.1:5000/predict', {
method: 'POST', headers: { 'Content-Type': 'application/json' },
body: JSON.stringify(inputData)
});
```

# 6. Habitability Scoring Logic

| Factor | Weight | Ideal Range | Max Score |
|---|---|---|---|
| Equilibrium Temperature | 35% | 220 – 300 K | 100 pts |
| Planet Radius | 25% | 0.8 – 1.5 R | 100 pts |
| Stellar Luminosity | 20% | 0.5 – 1.5 L | 100 pts |
| Semi-Major Axis | 10% | 0.7 – 1.4 AU | 100 pts |
| Orbital Period | 5% | 200 – 500 days | 100 pts |
| Stellar Mass | 5% | 0.7 – 1.3 M | 100 pts |

### Penalty Rules:

- Temperature > 500 K or < 100 K → score × 0.30 (70% reduction)
- Planet radius > 4.0 R → score × 0.20 (80% reduction — gas giant)
- Semi-major axis < 0.1 AU → score × 0.25 (75% reduction — too close to star)

| Score Range | Confidence Label | Final Prediction |
|---|---|---|
| 85 – 100% | Highly Habitable | Potentially Habitable |
| 70 – 84% | Moderately Habitable | Potentially Habitable |
| 50 – 69% | Possibly Habitable | Potentially Habitable |
| 30 – 49% | Unlikely Habitable | Not Habitable |
| 15 – 29% | Very Unlikely | Not Habitable |
| 0 – 14% | Not Habitable | Not Habitable |

# 7. Sample Planet Results

| Planet | Temp (K) | Radius (R) | Score | Confidence |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| Earth | 255 | 1.0 | ~95% | Highly Habitable |
| Kepler-452b | 265 | 1.6 | ~82% | Moderately Habitable |
| Kepler-442b | 233 | 1.34 | ~78% | Moderately Habitable |
| TRAPPIST-1e | 251 | 0.91 | ~70% | Possibly Habitable |
| Proxima Cen b | 234 | 1.08 | ~65% | Possibly Habitable |
| Mars | 210 | 0.53 | ~38% | Unlikely Habitable |
| Venus-like | 737 | 0.95 | ~12% | Very Unlikely |
| Hot-Jupiter-X | 800 | 11.2 | ~8% | Not Habitable |
| WASP-12b | 2500 | 15.4 | ~2% | Not Habitable |

## 8.
## Integration Testing

Complete flow tested: User Input → Frontend → Flask API → Scoring → Response → UI Display

| Test Case | Input | Expected Output | Result |
|---|---|---|---|
| Earth-like planet | Radius 1.0, Temp 255K | ~95% Highly Habitable | ■ Pass |
| Hot Jupiter | Radius 11.2, Temp 800K | < 10% Not Habitable | ■ Pass |
| Missing field | No planet_radius | 400 error with message | ■ Pass |
| Invalid input type | Radius: 'abc' | 400 error with message | ■ Pass |
| Health check | GET /health | status: healthy | ■ Pass |
| Rank endpoint | 3 planets array | Sorted by score desc | ■ Pass |
| API offline test | Flask stopped | Red dot + error message | ■ Pass |

## 9.
## Conclusion

| Module | Requirement | Status |
|---|---|---|
| Module 5 | Flask API with /predict + /rank endpoints | ■ Complete |
| Module 5 | ML model loaded from .pkl using joblib | ■ Complete |
| Module 5 | Input validation with meaningful error messages | ■ Complete |
| Module 5 | Clean structured JSON responses | ■ Complete |
| Module 6 | Responsive HTML/CSS/JS frontend with Bootstrap | ■ Complete |
| Module 6 | Input forms for all planetary parameters | ■ Complete |
| Module 6 | Frontend connected to backend via Fetch API | ■ Complete |
| Module 6 | Results shown with score, status, data table | ■ Complete |
| Module 6 | Client-side validation and error handling | ■ Complete |

ExoHabitAI — Built with Flask + Scikit-learn + HTML / CSS / JavaScript